S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Yutika Manekar

USN: CM24061

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

❖ **Aim:** Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

❖ **Objectives:**

1. Implement the Banker's Algorithm to manage resource allocation and prevent deadlocks.
2. Calculate the current resource need for each process based on allocated and maximum resources.
3. Determine a safe execution sequence for processes if resources are available.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

Banker's Algorithm in Operating System

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

- The Banker's Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- By keeping track of what resources each program needs and what's available, the banker algorithm makes sure that programs only get what they need in a safe order.

Why Banker's Algorithm is Named So?

The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. Let us assume that the bank has a certain amount of money Y. If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resource types.

1. **Available**
   - It is a 1-D array of size **'m'** indicating the number of available resources of each type.
   - Available[j] = k means there are **'k'** instances of resource type **Rj**
2. **Max**
   - It is a 2-d array of size '**n*m'** that defines the maximum demand of each process in a system.
   - Max[i, j] = k means process **Pi** may request at most **'k'** instances of resource type **Rj.**
3. **Allocation**
   - It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
   - Allocation[i, j] = k means process **Pi** is currently allocated **'k'** instances of resource type **Rj**
4. **Need**
   - It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
   - Need [i, j] = k means process **Pi** currently needs **'k'** instances of resource type **Rj**
   - Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Banker's Algorithm

1. Active := *Running U Blocked*
for k = 1 to r:
New_request[k] := Requested_resources[requesting_process, k]
2. Simulated_allocation := *Allocated_resources*
for k = 1 to r: // Compute projected allocation state
Simulated_allocation[requesting_process, k] :=
Simulated_allocation[requesting_process, k] + New_request[k]
3. Feasible := *true*
for k = 1 to r: // Check if projected allocation state is feasible
if Total_resources[k] < Simulated_total_alloc[k]:
Feasible := false
4. if Feasible = *true:*
while set Active contains a process P1 such that:
for all k, Total_resources[k] – Simulated_total_alloc[k] >=
Max_need[P1, k] – Simulated_allocation[P1, k]:
Delete P1 from Active
for k = 1 to r:
Simulated_total_alloc[k] :=
Simulated_total_alloc[k] – Simulated_allocation[P1, k]
5. if set Active is empty:
for k = 1 to r: // Delete the request from pending requests
Requested_resources[requesting_process, k] := 0
for k = 1 to r: // Grant the request
Allocated_resources[requesting_process, k] :=
Allocated_resources[requesting_process, k] + New_request[k]
Total_alloc[k] := Total_alloc[k] + New_request[k]

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
   Initialize: Work = Available
   Finish[i] = false; for i=1, 2, 3, 4….n
2. Find an i such that both
   2.1 Finish[i] = false
   2.2 Needi <= Work
   if no such i exists goto step (4)
3. Work = Work + Allocation[i]
   Finish[i] = true
   goto step (2)
4. if Finish [i] = true for all i then the system is in a safe state.

Resource-Request Algorithm

Let Requesti be the request array for process Pi. Requesti [j] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1. If Requesti <= Needi

Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.

2. If Requesti <= Available

Goto step (3); otherwise, Pi must wait, since the resources are not available.

3. Have the system pretend to have allocated the requested resources to process Pi by modifying the state as

follows:

Available = Available – Requesti

Allocationi = Allocationi + Requesti

Needi = Needi– Requesti

Example: Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Unsafe State in The Bankers Algorithm

In the context of the Banker's Algorithm, an **unsafe state** refers to a situation in which, all processes in a system currently hold resources within their maximum needs, there is no guarantee that the system can avoid a deadlock in the future. This state is contrasted with a **safe state**, where there exists a sequence of resource allocation and deallocation that guarantees that all processes can complete without leading to a deadlock.

Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Example of Unsafe State

Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:

- The available instances of the resource are: 1
- The current allocation of resources to processes is:
  - P1: 2
  - P2: 3
  - P3: 1
- The maximum demand (maximum resources each process may eventually

request) is:

- P1: 4
- P2: 5
- P3: 3

In this situation:

Need = Maximum Demand – Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

❖ CODE

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define NUM_PROCESSES 5
#define NUM_RESOURCES 4

// Process names
char* process_names[] = {"Google Drive", "Firefox", "Word Processor", "Excel",
    "PowerPoint"};

// Resource names
char* resource_names[] = {"Printer", "ROM", "Hard Disk", "RAM"};

// Function to calculate need matrix
void calculateNeed(int need[NUM_PROCESSES][NUM_RESOURCES],
                   int max[NUM_PROCESSES][NUM_RESOURCES],
                   int alloc[NUM_PROCESSES][NUM_RESOURCES]) {
   for(int i = 0; i < NUM_PROCESSES; i++) {
      for(int j = 0; j < NUM_RESOURCES; j++) {
         need[i][j] = max[i][j] - alloc[i][j];
      }
   }
}
```

```
// Function to check if system is in safe state
bool isSafeState(int available[NUM_RESOURCES],
                 int max[NUM_PROCESSES][NUM_RESOURCES],
                 int alloc[NUM_PROCESSES][NUM_RESOURCES],
                 int need[NUM_PROCESSES][NUM_RESOURCES]) {

    int work[NUM_RESOURCES];
    bool finish[NUM_PROCESSES];
    int safeSequence[NUM_PROCESSES];
    int count = 0;

    // Initialize work with available resources
    for(int i = 0; i < NUM_RESOURCES; i++) {
        work[i] = available[i];
    }

    // Initialize finish array to false
    for(int i = 0; i < NUM_PROCESSES; i++) {
        finish[i] = false;
    }

    // Find safe sequence
    while(count < NUM_PROCESSES) {
        bool found = false;

        for(int p = 0; p < NUM_PROCESSES; p++) {
            if(finish[p] == false) {
                // Check if need <= work for all resources
                bool canExecute = true;
                for(int j = 0; j < NUM_RESOURCES; j++) {
                    if(need[p][j] > work[j]) {
                        canExecute = false;
                        break;
                    }
                }
                if(canExecute) {
                    // Add allocated resources back to work
                    for(int j = 0; j < NUM_RESOURCES; j++) {
                        work[j] += alloc[p][j];
                    }
                    safeSequence[count++] = p;
                    finish[p] = true;
                    found = true;

                    printf("Process %s can execute\n", process_names[p]);
                }
            }
        }

        if(!found) {
```

```
            printf("\nNo safe sequence exists!\n");
            return false;
        }
    }

    // Print safe sequence
    printf("\n*** System is in SAFE STATE ***\n");
    printf("Safe Sequence: ");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        printf("%s", process_names[safeSequence[i]]);
        if(i < NUM_PROCESSES - 1) {
            printf(" -> ");
        }
    }
    printf("\n");

    return true;
}

// Function to display resource allocation table
void displayTable(int alloc[NUM_PROCESSES][NUM_RESOURCES],
                  int max[NUM_PROCESSES][NUM_RESOURCES],
                  int need[NUM_PROCESSES][NUM_RESOURCES],
                  int available[NUM_RESOURCES]) {
    printf("\n==================================================");
    printf("\nRESOURCE ALLOCATION TABLE");
    printf("\n==================================================\n");

    // Print header
    printf("%-15s", "Process");
    for(int j = 0; j < NUM_RESOURCES; j++) {
        printf("%12s", resource_names[j]);
    }
    printf("\n--------------------------------------------------\n");

    // Print Allocation
    printf("\n%-15s", "ALLOCATION:");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        if(i == 0) printf("\n%-15s", process_names[i]);
        else printf("\n%-15s", "");

        for(int j = 0; j < NUM_RESOURCES; j++) {
            printf("%12d", alloc[i][j]);
        }
    }

    // Print Maximum
    printf("\n\n%-15s", "MAXIMUM:");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        if(i == 0) printf("\n%-15s", process_names[i]);
        else printf("\n%-15s", "");
```

```
        for(int j = 0; j < NUM_RESOURCES; j++) {
            printf("%12d", max[i][j]);
        }
    }

    // Print Need
    printf("\n\n%-15s", "NEED:");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        if(i == 0) printf("\n%-15s", process_names[i]);
        else printf("\n%-15s", "");

        for(int j = 0; j < NUM_RESOURCES; j++) {
            printf("%12d", need[i][j]);
        }
    }

    // Print Available
    printf("\n\n%-15s", "AVAILABLE:");
    printf("\n%-15s", "");
    for(int j = 0; j < NUM_RESOURCES; j++) {
        printf("%12d", available[j]);
    }
    printf("\n==================================================\n");
}

// Function to input allocation matrix
void inputAllocation(int alloc[NUM_PROCESSES][NUM_RESOURCES]) {
    printf("\nEnter ALLOCATION Matrix:\n");
    printf("-------------------------\n");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        printf("For %s:\n", process_names[i]);
        for(int j = 0; j < NUM_RESOURCES; j++) {
            printf("  %s: ", resource_names[j]);
            scanf("%d", &alloc[i][j]);
        }
        printf("\n");
    }
}

// Function to input maximum matrix
void inputMax(int max[NUM_PROCESSES][NUM_RESOURCES]) {
    printf("\nEnter MAXIMUM Matrix:\n");
    printf("----------------------\n");
    for(int i = 0; i < NUM_PROCESSES; i++) {
        printf("For %s:\n", process_names[i]);
        for(int j = 0; j < NUM_RESOURCES; j++) {
            printf("  %s: ", resource_names[j]);
            scanf("%d", &max[i][j]);
        }
        printf("\n");
```

```
    }
}

// Function to input available resources
void inputAvailable(int available[NUM_RESOURCES]) {
    printf("\nEnter AVAILABLE Resources:\n");
    printf("-------------------------\n");
    for(int j = 0; j < NUM_RESOURCES; j++) {
        printf("%s: ", resource_names[j]);
        scanf("%d", &available[j]);
    }
}

int main() {
    int alloc[NUM_PROCESSES][NUM_RESOURCES];
    int max[NUM_PROCESSES][NUM_RESOURCES];
    int need[NUM_PROCESSES][NUM_RESOURCES];
    int available[NUM_RESOURCES];

    printf("\n==================================================");
    printf("\n    BANKER'S ALGORITHM IMPLEMENTATION");
    printf("\n==================================================\n");

    // Input data
    inputAllocation(alloc);
    inputMax(max);
    inputAvailable(available);

    // Calculate need matrix
    calculateNeed(need, max, alloc);

    // Display the table
    displayTable(alloc, max, need, available);

    // Check for safe state
    printf("\nChecking for Safe State...\n");
    printf("==========================\n");

    if(isSafeState(available, max, alloc, need)) {
        printf("\n✓ The system can execute all processes safely!\n");
    } else {
        printf("\n✗ The system is in UNSAFE state!\n");
        printf("  Deadlock may occur if all processes request maximum resources.\n");
    }

    printf("\n==================================================\n");
    printf("Program completed successfully!\n");
    printf("==================================================\n");

    return 0;
  }
```

❖ Output:

```
ubuntu@ubuntu:~$ nano bankers_algo.c
ubuntu@ubuntu:~$ gcc bankers_algo.c -o bankers_algo
ubuntu@ubuntu:~$ ./bankers_algo

==============================================
    BANKER'S ALGORITHM IMPLEMENTATION
==============================================

Enter ALLOCATION Matrix:
-----------------------
For Google Drive:
  Printer: 1
  ROM: 0
  Hard Disk: 1
  RAM: 2

For Firefox:
  Printer: 2
  ROM: 1
  Hard Disk: 0
  RAM: 1

For Word Processor:
  Printer: 0
  ROM: 2
  Hard Disk: 1
  RAM: 0

For Excel:
  Printer: 1
  ROM: 1
  Hard Disk: 2
  RAM: 1

For PowerPoint:
  Printer: 0
  ROM: 0
  Hard Disk: 1
  RAM: 2
```

```
Enter MAXIMUM Matrix:
--------------------
For Google Drive:
  Printer: 3
  ROM: 2
  Hard Disk: 2
  RAM: 4

For Firefox:
  Printer: 4
  ROM: 3
  Hard Disk: 3
  RAM: 3

For Word Processor:
  Printer: 2
  ROM: 4
  Hard Disk: 2
  RAM: 2

For Excel:
  Printer: 3
  ROM: 3
  Hard Disk:
4
  RAM: 3

For PowerPoint:
  Printer: 2
  ROM: 2
  Hard Disk: 3
  RAM: 4
```

```
Enter AVAILABLE Resources:
-------------------------
Printer: 2
ROM: 3
Hard Disk: 2
RAM: 2

==============================================
RESOURCE ALLOCATION TABLE
==============================================
Process             Printer         ROM   Hard Disk         RAM
----------------------------------------------

ALLOCATION:
Google Drive             1           0           1           2
                         2           1           0           1
                         0           2           1           0
                         1           1           2           1
                         0           0           1           2

MAXIMUM:
Google Drive             3           2           2           4
                         4           3           3           3
                         2           4           2           2
                         3           3           4           3
                         2           2           3           4

NEED:
Google Drive             2           2           1           2
                         2           2           3           2
                         2           2           1           2
                         2           2           2           2
                         2           2           2           2

AVAILABLE:
                         2           3           2           2
==============================================
```

```
Checking for Safe State...
==========================
Process Google Drive can execute
Process Firefox can execute
Process Word Processor can execute
Process Excel can execute
Process PowerPoint can execute

*** System is in SAFE STATE ***
Safe Sequence: Google Drive -> Firefox -> Word Processor -> Excel -> PowerPoint

✓ The system can execute all processes safely!

==============================================
Program completed successfully!
==============================================
ubuntu@ubuntu:~$
```

- **Conclusion**: In this practical, we conclude that the Banker's Algorithm effectively ensures safe resource allocation by preventing deadlocks. By calculating the current resource need of each process and checking for a safe execution sequence, we can determine whether all processes can complete execution without system instability.

❖ Discussion Questions:

1. **What is the purpose of the Banker's Algorithm?**
2. **How do you calculate the current need of each process?**
3. **What are the conditions for a system to be in a safe state?**
4. **What happens if no safe sequence exists in the Banker's Algorithm?**
5. **What are the key inputs required to implement the Banker's Algorithm?**

❖ References:

https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/

https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26